Lctr 16: Shift and Rotation Instructions, Op2, and Applications

Jianhua Liu

Spring 2026

==Topics==

Core concepts and applications:

- Shift operations in C.
- ASM shift instructions having C counterparts.
  - Useful in combination with bitwise operations.
  - Implementation of Q31 multiplication in ASM with help of shift.
- ASM rotation instructions having no C counterparts (skipped in class).
- Barrel shifter, Op2 (the flexible second operand), and applications.
  - Improved implementation of Q31 multiplication using Op2.
  - Fast Q31 scaling.
  - Special operations.

Demo project:

- Various ASM functions to implement Q31 multiplication in different approaches.
- Various ASM functions for shifting 64-bit numbers (skip in class).

16.1 Introduction

In this lecture, we cover shift and rotation instructions in ASM. We will also look at how Op2 is implemented with the help of shifting, which can be used for additional functionalities. We will also see how to use these instructions and added functionalities in real applications.

Before proceeding, let us take a look at the shift operators in C first.

16.1.1 Shift operations in C

C has the following shift operators:

Precedence	Operator	Description	Associativity
5	<<, >>	Bitwise left and right shift	Left to right

- For the left-shift operation, 0s come in from LSB side. (8-bit examples)
  - Shifting n bits to the left, the value multiplied by 2^n. Expl: `0x3F << 1` ==>
  - There may be *overflow* for the shifted result. Expl: (int) `0x7F << 1` ==>
- For the right-shift operation, 0s or 1s come in from MSB side, depending on the data type:
  - For unsigned variables, always use 0. Expl: `0x7F >> 1` ==> ____, `0x80>> 1` ==> ____
  - For signed variables, repeat the MSB. Expl: `0x7F >> 1` ==> ____, `0x80>> 1` ==> ____
- The shift operation can be used to variables of different number of bits—from 8 to 64.

The code below shows two equivalent expressions on the right-hand side.

```
X = A + B >> 2 << 2;
Y = ((A + B) >> 2) << 2;
```

While it may be possible to use the more concise expression, like the first one, we generally prefer to avoid it in our code. This is because such expressions can make the code less readable and more difficult for others (or ourselves) to understand.

16.1.2 ASM shift and rotation instructions

See the topics on the first page.

16.2 Demo code

The demo project, `fg1s_shift_instructions`, has two main parts.

- Various ASM functions to implement Q31 multiplication in different approaches.
  - `int mp_mult_q31_fast_s(int A, int B)` for fast implementation.
  - `int mp_mult_q31_accu_s(int A, int B)` for accurate implementation.
  - `int mp_mult_q31_s(int A, int B)` for both fast and accurate implementation.
- Various ASM functions for shifting 64-bit numbers. As a 32-bit system, there are no instructions to perform shift operations on 64-bit numbers. Yet, we can use functions to implement these operations, as illustrated below:
  - `int64_t mp_64bit_lsl_0_32_s(int64_t A, int n)` for 0 to 32 bit logic shift-to-left for both `uint64_t` and `int64_t`.
  - `uint64_t mp_64bit_lsr_0_32_s(uint64_t A, int n)` for 0 to 32 bit logic shift-to-right for `uint64_t`.
  - `int64_t mp_64bit_asr_0_32_s(int64_t A, int n)` for 0 to 32 bit arithmetic shift-to-right for `int64_t`.

We will only cover the Q31-related demo code in the class lecture. Students with interests in 64-bit operations or the code implementations are suggested to read the code directly.

16.3 32-bit shift instructions with corresponding operations in C

There are three 32-bit shift instructions with corresponding C shift operation, as listed in the table below.

Instruction	Description
LSL {Rd,} Rn, Rm (or `#n`)	Logic shift to left: Rd <– (Rn shift to left by Rm (or `#n`) bits)
LSR {Rd,} Rn, Rm (or `#n`)	Logic shift to right: Rd <– (Unsigned value in Rn shift to right by Rm (or `#n`) bits)
ASR {Rd,} Rn, Rm (or `#n`)	Arithmetic shift to right: Rd <– (Signed value in Rn shift to right by Rm (or `#n`) bits)

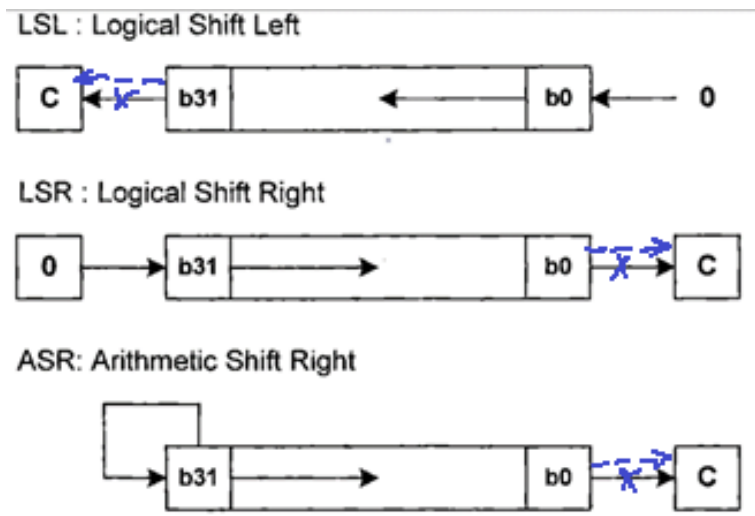

Figure 1: Bitwise shift and the C flag.

- All of these instructions have the S version, which will update the C flag.
- LSL is used for both unsigned and signed shifting.
- LSR is used for unsigned (logic values) shift only.
- ASR is used for signed shifting only.
- An advantage of these instructions over C's operator is that we can use the C flag, as will be illustrated using an example for implementing the Q31 multiplication below.

16.3.1 Basic examples with shift instructions

Here, let's consider a number of small examples of the shift instructions.

```
lsl  r1, r2       @ r1 = r1 << r2
lsl  r1, #3       @ r1 = r1 << 3
lsl  r1, r2, #3   @ r1 = r2 << 3
lsl  r1, r2, r3   @ r1 = r2 << r3
lsls r1, r2, r3   @ r1 = r2 << r3, N, Z, and C flags updated.
lsr  r1, r2, r3   @ r1 = r2 >> r3 for unsigned
asr  r1, r2, r3   @ r1 = r2 >> r3 for signed
```

16.3.2 Implementation of Q31 multiplication with help of shifting

As a more complex example, let us discuss the ASM implementation of Q31 multiplication using multiplication and shift instructions.

First, let's recall the C implementation:

```
// C standard implementation of Q31 multiplication:
q31_t mp_mult_q31(q31_t A, q31_t B) {
    q63_t prod = (q63_t)A * B;
    return (q31_t) (prod >> 31);
}
```

Note that `q31_t` and `q63_t` are redefinitions of `int32_t` and `int64_t`, respectively.

There are *two* implementations of the above C code in ASM, as discussed below.

16.3.2.1 Fast implementation of Q31 multiplication in ASM

The **fast** approach can be done with TWO instructions (not considering the return instruction); yet, it will sacrifice the accuracy by *one bit*:

```
@ int mp_mult_q31_fast_s(int A, int B)
mp_mult_q31_fast_s:
    smull   r1, r0, r0, r1
    lsl     r0, #1
    bx      lr
```

(Line 3) `smull r1, r0, r0, r1` performs the 32-bit by 32-bit to 64-bit multiplication using the signed long multiplication instruction, which is equivalent to type-casting r0 to 64-bit first. The multiplication result is saved in R0 (high word, HW) and R1 (low word, LW), and the entire 64-bit double word is shown below.

$$\overbrace{\boxed{b_{63} \mid \quad \ldots \quad \mid b_{33} \mid b_{32}}}^{\text{HW = R0}}\overbrace{\boxed{b_{31} \mid \quad \ldots \quad \mid b_{1} \mid b_{0}}}^{\text{LW = R1}}$$

Now, to shift this double word to the right by 31 bit, we should have

$$\overbrace{\boxed{b_{63} \mid \quad \ldots \quad \mid b_{63} \mid b_{63}}}^{\text{HW = R0}}\overbrace{\boxed{b_{62} \mid \quad \ldots \quad \mid b_{32} \mid b_{31}}}^{\text{LW = R1}}$$

(Line 4) `lsl r0, #1` is a fast approach for implementing this shift, which leads to

$$\text{R0} = \boxed{b_{62} \mid \quad \ldots \quad \mid b_{32} \mid 0}$$

16.3.2.2 Accurate implementation of Q31 multiplication in ASM

The **accurate** approach will be able to have b_{31} back; yet it will sacrifice the speed a little, using FOUR instructions, as shown below:

```
@ int mp_mult_q31_accu_s(int A, int B)
mp_mult_q31_accu_s:
    smull   r1, r0, r0, r1
    lsl     r0, #1
    lsls    r1, #1
    adc     r0, #0
    bx      lr
```

We can see that we have just added two more instructions to the *fast* version in the above *accurate* code.

(Line 5) `lsls r1, #1` moves b_{31} out of R1 into the C flag bit of APSR.

(Line 6) `adc r0, #0` add this C flag, which is b_{31}, to R0 to obtain:

$$\text{R0} = \boxed{b_{62} \mid \quad \ldots \quad \mid b_{32} \mid b_{31}}$$

16.3.2.3 Testing ASM implementations

To test the above ASM implementations, we use the following test functions:

```
void test_mult_q31_fast_s_equal_should_pass(void) {
    int I_A = 1 << 28;
    int I_B = 1 << 28;
    int exp = ((int64_t)I_A * I_B) >> 31;
    int act = mp_mult_q31_fast_s(I_A, I_B);
    TEST_ASSERT_EQUAL_INT32(exp, act);
}

void test_mult_q31_fast_s_not_equal_should_pass(void) {
    int I_A = 0x7FFFFFFF;
    int I_B = 0x80000000;   // q31 code for -1.0
    int exp = -0x7FFFFFFF;
    int act = mp_mult_q31_fast_s(I_A, I_B);
    TEST_ASSERT_FALSE(exp == act);
}
```

```
void test_mult_q31_accu_s_should_pass(void) {
    int I_A = 1 << 28;
    int I_B = 1 << 28;
    int exp = ((int64_t)I_A * I_B) >> 31;
    int act = mp_mult_q31_accu_s(I_A, I_B);
    TEST_ASSERT_EQUAL_INT32(exp, act);
}

void test_mult_q31_accu_s_should_also_pass(void) {
    int I_A = 0x7FFFFFFF;
    int I_B = 0x80000000;
    int exp = -0x7FFFFFFF;
    int act = mp_mult_q31_accu_s(I_A, I_B);
    TEST_ASSERT_EQUAL_INT32(exp, act);
}
```

16.4 32-bit rotate instructions without corresponding C operations

These instructions are not as useful as the shift instructions and will be skipped here in the slides. Students with interests in these instructions are suggested to read the corresponding section of the Collection.

16.5 Barrel shifter and Op2

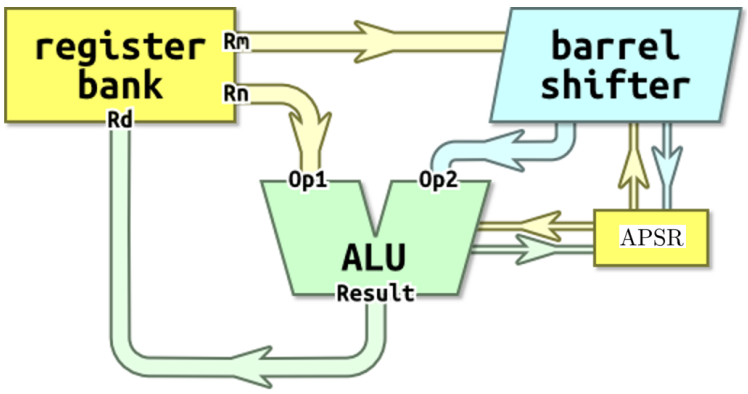

Figure 2: ALU, barrel shifter, and flexible operand 2 (Op2).

There are three different situations for the second operand for **many** ARM instructions, which lead to the name of Op2, the flexible operand 2:

- An immediate number.
- A register without shift.
- A register with shift. **For this case, we cannot omit the destination register even if it is the same as the first source register**. Otherwise, we will see a syntax error.

16.5.1 Special operations using Op2-based operations

With the barrel shifter, we can perform the following operations easily:

- `r2= r1+ 2*r0`
- `r2= r0/4- r1`

which are performed by the following instructions respectively:

```
add r2, r1, r0, lsl #1  @ Op2 = r0, lsl #1
rsb r2, r1, r0, asr #2  @ Op2 = r0, asr #2
```

Note that we need to be careful with the intended number representation—unsigned or signed. For example, while the first one shown above can be used for both unsigned and signed, the second can only be used for signed as ASR repeat the MSB of R0.

We use a number of examples to better illustrate these operations.

Example 1. Express R1 as a function of R0 and determine the data type of R0 and R1 for the following instructions.

```
(a) add r1, r0, r0, lsr #1
(b) sub r1, r0, r0, asr #2
(c) rsb r1, r0, r0, asr #2
```

Example 2. Express R1 as a function of R0 and determine the data type of R0 and R1 for the following instructions.

```
(a) add r1, r0, r0, lsl #1
(c) rsb r1, r0, r0, lsl #2
(b) sub r1, r0, r0, lsl #2
```

16.5.2 Implementation of fast Q31 scaling for special cases

Assume a Q31 number is in R0 already. **Multiplying the number by 5/4 = 1.25** and saving the result back to R0 can be done in the following way:

```
add  r0, r0, r0, asr #2
```

which can be understood as

```
r0 = r0 + (r0 >> 2) = r0 + r0 / 4 = 5 * r0 / 4.
```

Similarly, we can **scale the number by 3/4 = 0.75** using the following way:

```
sub r0, r0, r0, asr #2
```

which can be understood as

```
r0 = r0 - (r0 >> 2) = r0 - r0 / 4 = 3 * r0 / 4.
```

We have the following notes about these scaling:

- They only works for the special cases that the denominator is the power of 2, say 4 and 8.
- This implementation is faster than the conventional multiplication-based approach. For example, if we want to multiply a Q31 number by 1.25 using the conventional way, we have to do it in two steps:
  - First, multiply the number by 1.25 / 2 = 0.625.
  - Then, multiply the number by 2 (implemented via left-shifting the number by one bit).
- An additional advantage of doing so is that this kind of operations remove the possible overflow and keep the accuracy elegantly. For example, the above multiplication of 1.25 using Q31 multiplication and shift to the right by one bit will leave the LSB always being 0. With this Op2 based scaling, we don't have this issue.

16.5.3 Implementation of the improved Q31 multiplication

In the implementation of `mp_mult_q31_accu_s` before, we have used

```
    lsls    r1, #1
    adc     r0, #0
```

to replace the 0 at the LSB of R0 by the MSB of R1. Now, with Op2, we can have a more efficient approach, shown below together with the other code:

```
@ int mp_mult_q31_s(int A, int B)
mp_mult_q31_s:
    smull   r1, r0, r0, r1
    lsl     r0, #1
    add     r0, r0, r1, lsr #31
    bx      lr
```

To understand the above code, we need to know that `r1, lsr #31` shifts the value of R1 as an

unsigned to the right by 31 bit, and this is effectively

$$\boxed{0 \mid \quad \ldots \quad \mid 0 \mid b_{31}}$$

This is what we need to add to R0, which has the value of

$$\boxed{b_{62} \mid \quad \ldots \quad \mid b_{32} \mid 0}$$

before the addition.

Note that the above `add r0, r0, r1, lsr #31` instruction can be replaced by `orr r0, r0, r1, lsr #31`, the bitwise OR instruction that we will discuss in a later lecture.

16.6 Additional exercises

Exercise 1. Express R1 as a function of R0 and determine the intended data type of R0 and R1.

```
(a) add r1, r0, r0, lsr #3
(b) add r1, r0, r0, lsl #3
(c) sub r1, r0, r0, asr #3
(d) sub r1, r0, r0, lsl #3
(e) rsb r1, r0, r0, asr #3
(f) rsb r1, r0, r0, lsl #3
```

16.7 Additional reading